Universidade do Minho
Escola de Engenharia
Departamento de Informática

KNIME

Pre-Processing + Linear Regression

MESTRADO EM ENGENHARIA DE TELECOMUNICAÇÕES E INFORMÁTICA
Inteligência Artificial para as Telecomunicações
2025/2026

ISLab

Objectives

- The primary goal of this task is **to create a comprehensive and efficient workflow** tailored specifically for pre-processing the dataset to develop a **logistic regression model** for predicting survival on the Titanic dataset .

- The pre-processing phase is critical for preparing raw data, ensuring it is clean, well-structured, and suitable for further analysis or modeling.

ISLab

Pre-processing

- Data Cleaning: Identifying and handling missing values, correcting inconsistencies, and removing duplicates to ensure data quality.
- Data Transformation: Applying necessary transformations, such as scaling, normalization, and encoding categorical variables, to standardize the dataset.
- Data Integration: Combining or merging multiple datasets to create a unified dataset that is ready for analysis.
- Feature Engineering and Selection: Creating new features, selecting important ones, or reducing dimensionality to improve model performance and computational efficiency.

ISLab

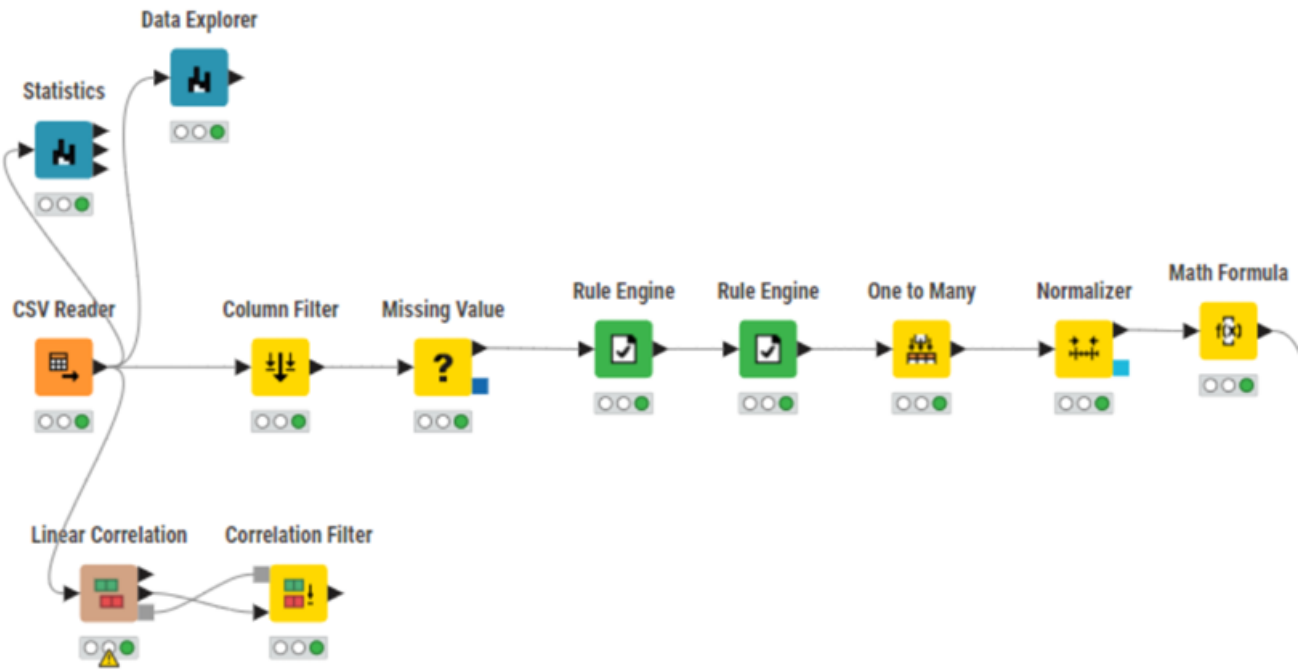
Dataset

12 attributes

PassengerId: An identification number for each passenger;
Survived: This is the target variable (0 = The passenger did not survive, 1 = The passenger survived);
Pclass: Passenger class (1 = Upper class, 2 = Middle class, 3 = Lower class);
Name: Full name of the passenger;
Sex: Gender of the passenger (male or female);
Age: Age of the passenger in years (with some missing values);
SibSp: Number of siblings/spouses aboard the Titanic;
Parch: Number of parents/children aboard the Titanic;
Ticket: Ticket number;
Fare: The ticket fare the passenger paid;
Cabin: Cabin number (with many missing values);
Embarked: Port of embarkation (with C = Cherbourg, Q = Queenstown, S = Southampton).

ISLab

Pre-processed workflow

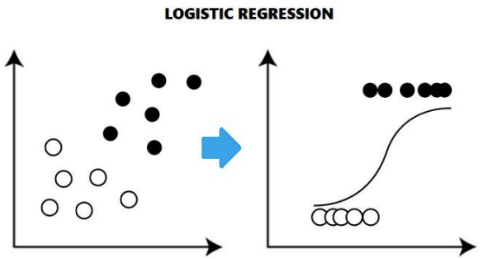


ISLab

KNIME Analytics Platform

Goal:

- Understand **Logistic Regression**
- Develop a **Logistic Regression model** capable of predicting the "***Survived***" feature withing the Titanic dataset.
- Analyze the measures used to **evaluate the model**



Logistic Regression

Logistic Regression is a statistical model used for binary **classification** tasks, where the goal is to predict the probability of **one of two outcomes**, i.e., ”yes” or ”no”. Instead of fitting a straight line like in linear regression, logistic regression applies a logistic function to produce a probability between 0 and 1.

ISLab

Logistic Regression

To develop a logistic regression model for predicting survival on the Titanic dataset, we can use logistic regression to determine the probability that a passenger survived or did not survive based on several features such as age, sex, passenger class, and more.

Here's how:

- **Binary Classification:** Since the "Survived" feature in the Titanic dataset is binary (1 for survived, 0 for did not survive), logistic regression is well-suited for modeling this kind of outcome.
- **Probability Prediction:** Logistic regression will estimate the probability of each passenger surviving based on their characteristics. This probability output is between 0 and 1, thanks to the logistic function, which maps inputs into a probability.
- **Decision Boundary:** By setting a threshold (often 0.5), we can use this probability to classify each passenger as likely to survive (1) or not (0).
- **Feature Influence:** The logistic regression model will assign coefficients to each input feature (i.e., age, sex, class) to indicate how they impact the likelihood of survival. Positive coefficients increase the probability of survival, while negative ones decrease it.

Logistic Regression

Open the workflow from the previous class, press Shift and, with your mouse, hover through all nodes until the ***Partitioning*** node. After that, press **Ctrl+C.**

Create a **new workflow** and press **Ctrl+V** to paste all the **selected** nodes.

Logistic Regression

To successfully implement a Logistic Regression model, some **changes** need to be made in the ***pre-processing*** nodes that we just pasted.

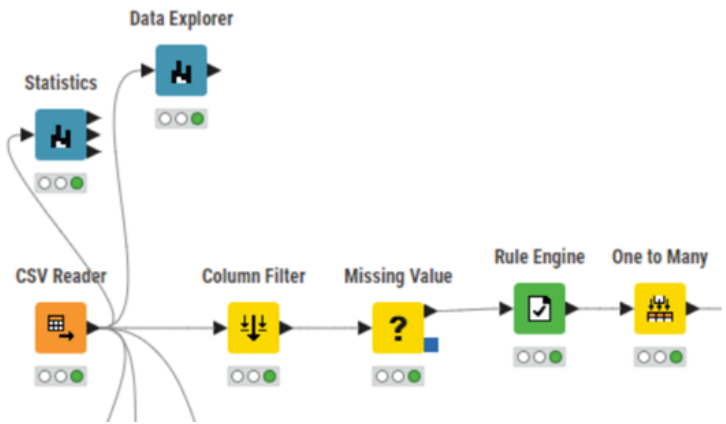
Logistic Regression

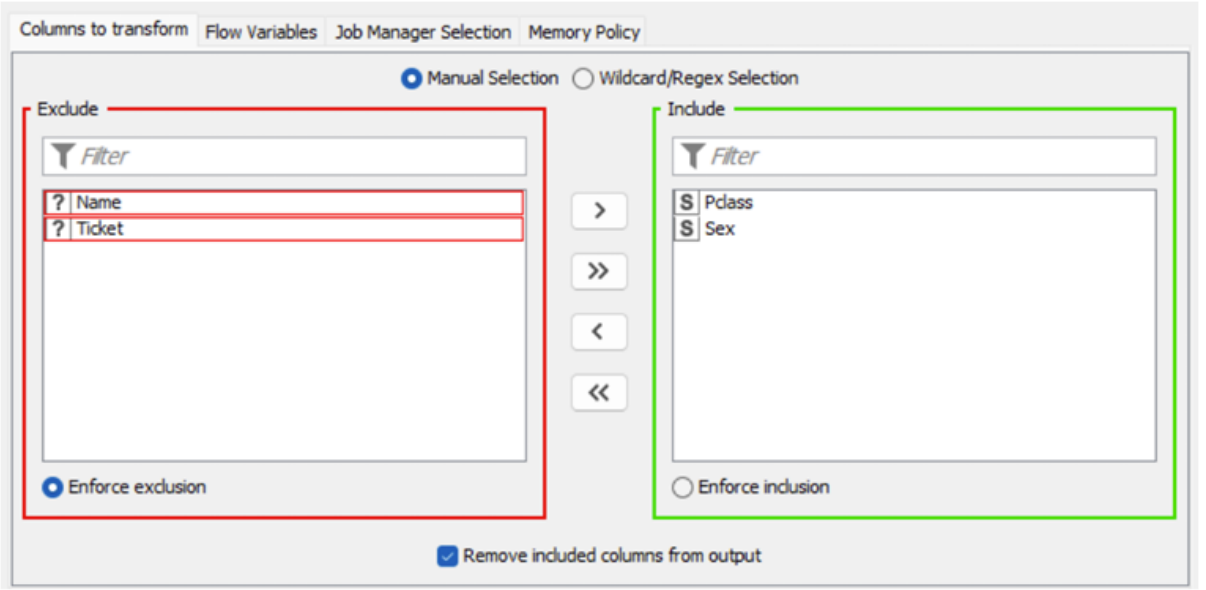
To improve the logistic regression model:

- **delete** the initial **Rule Engine** node that maps the values of the Sex attribute to binary values (i.e., male as 1 and female as 0).
- Instead, include this attribute in the **One to Many** node, which **encodes categorical** attributes as **separate binary** columns.

Mapping Sex to binary values (0 or 1) in the Rule Engine node can create an unintended assumption that one category has more significance than the other. By using one-hot encoding, each category (i.e., Male and Female) is represented independently, removing any notion of ordering or preference between the categories. Additionally, in logistic regression, one-hot encoding makes it easier to interpret the effect of each category independently, as each encoded category gets its own coefficient. This way, we can see the individual impact of being male or female on the survival prediction.

Logistic Regression





After removing the first Rule Engine node, open the configuration panel of the **One to Many** node and **include** the **Sex** attribute.

Logistic Regression

After running the node, you can see that **two** new columns were generated for each value of the Sex attribute: a column for ”**female**” and a column for ”**male**”, each containing **binary values**.

Third *Number (intege...*	First *Number (intege...*	Second *Number (intege...*	male *Number (intege...*	female *Number (intege...*
1	0	0	1	0
0	1	0	0	1
1	0	0	0	1
0	1	0	0	1

Logistic Regression

Delete the **Math Formula** node that derives the ***FamilySize*** variable.

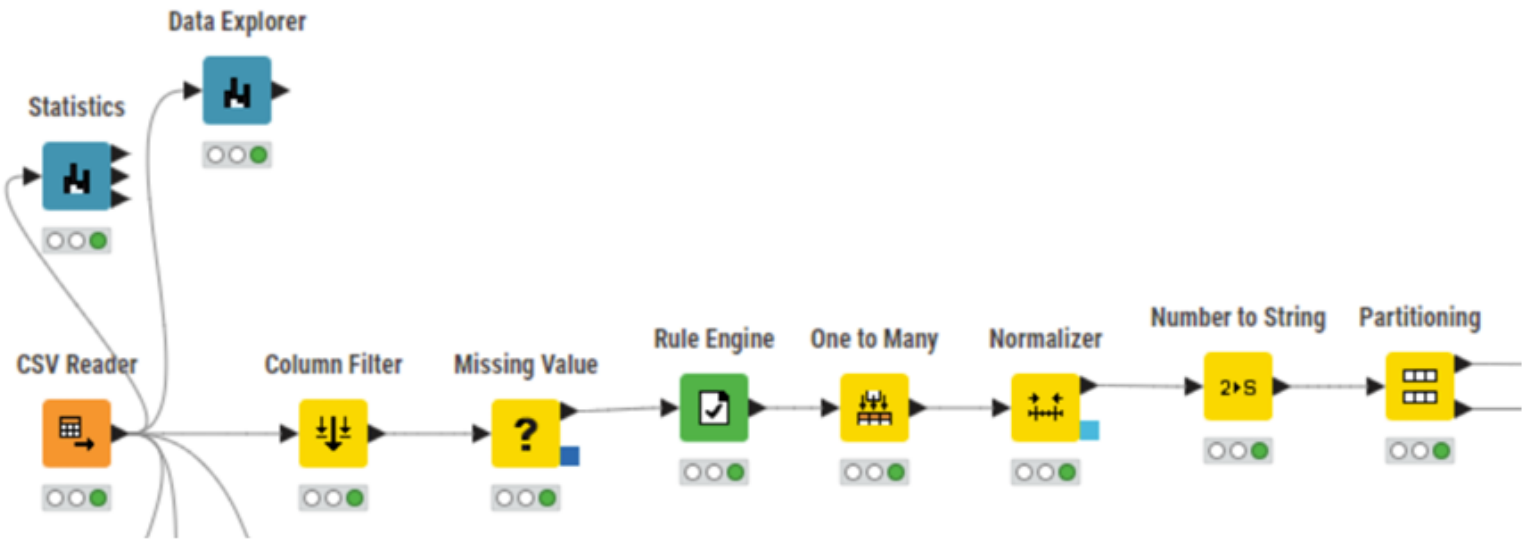
Logistic regression is a linear model that can **struggle with complexity**. Including too many variables increases the dimensionality of the dataset and can complicate the decision boundary the model is trying to learn.

Thus, simplifying the model by removing FamilySize can lead to better performance, as the logistic regression can better approximate the true relationship between features and the target variable.

Logistic Regression

In its place, we will be inserting the **Number to String** node.

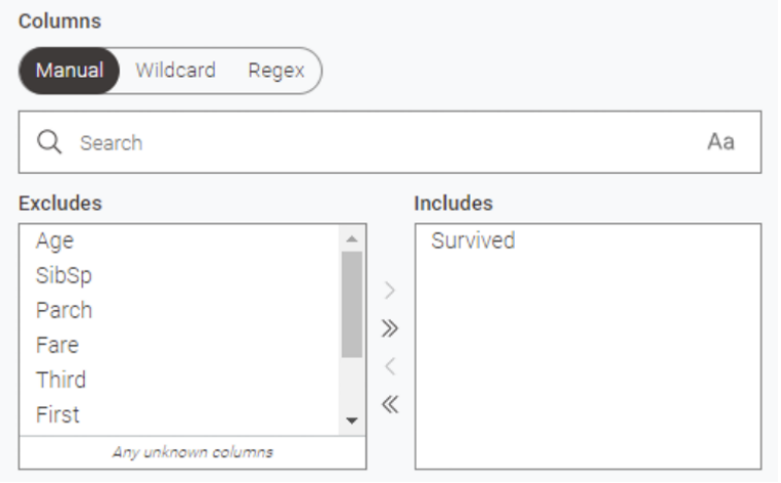
In the Node Repository, search for the **Number to String** node. Insert it **between the Normalizer node and the Partitioning node.**



Logistic Regression

The Number to String node will be applied to the **Survived** attribute to **convert** it from a **numeric** type to a **string** type.

Using ”0” and ”1” (the values of the Survived attribute) as strings allows us to treat them as categorical labels, ensuring that logistic regression interprets them as **distinct classes rather than numeric values.** This helps avoid any unintended mathematical operations and clarifies to the model that these values represent categories (survived or not) rather than a continuum.

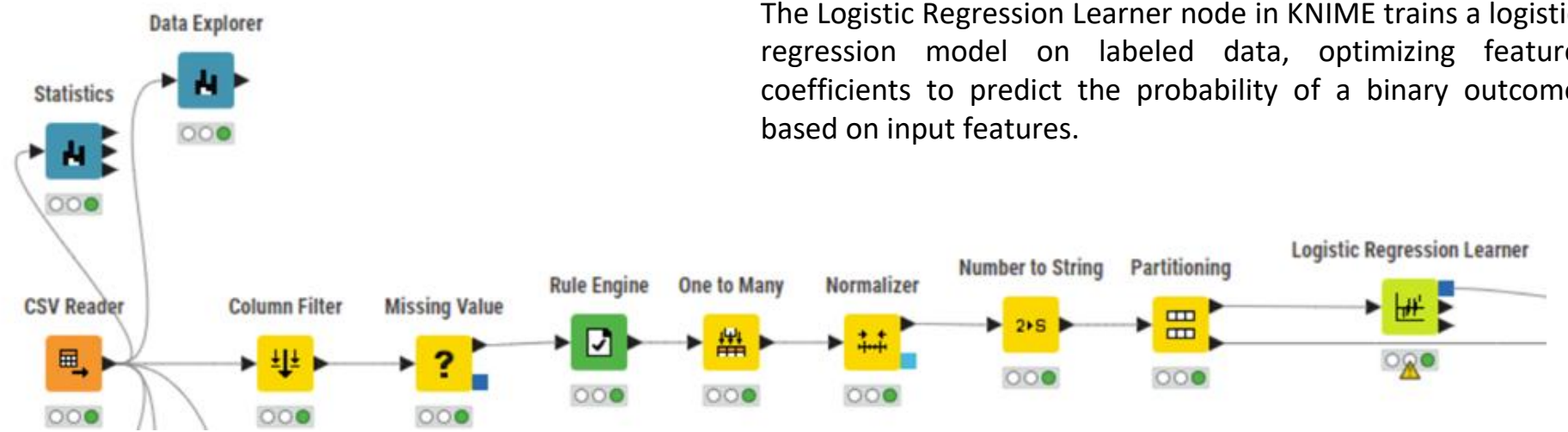


Open the Number to String node’s configuration panel. Include the Survived attribute and run the node.

Logistic Regression

In the Node Repository, search for the **Logistic Regression Learner** node.

Connect the **output** port of the **Partitioning** node to the **input** port of the **Logistic Regression Learner** node.



The Logistic Regression Learner node in KNIME trains a logistic regression model on labeled data, optimizing feature coefficients to predict the probability of a binary outcome based on input features.

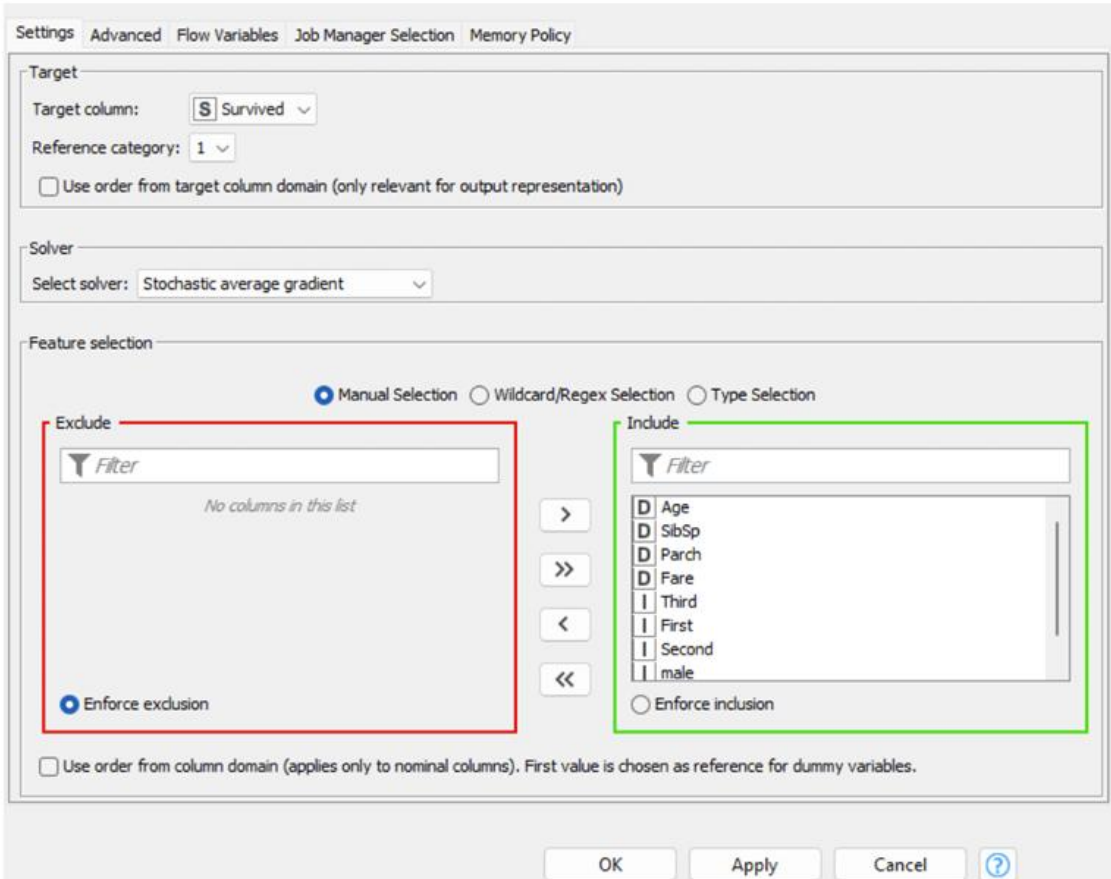
Logistic Regression

Open the configuration panel of the

Logistic Regression Learner node.

Make sure the **target** column is **Survived**!

Run the node.



Logistic Regression

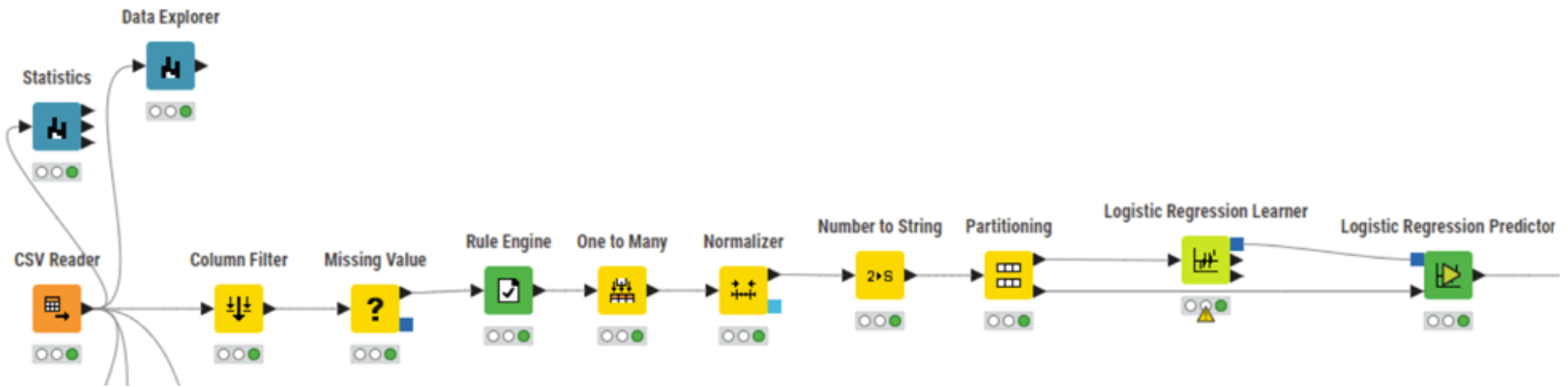
In the Node Repository, search for the **Logistic Regression Predictor** node.

Connect the **output** port of the **Logistic Regression Learner** node to the input port of the **Logistic Regression Predictor** node. Connect the **second output** port of the **Partitioning** node to the **second input** port of the **Logistic Regression Predictor** node.

The Logistic Regression Predictor node in KNIME uses the trained logistic regression model to generate predicted probabilities and class labels for new data, determining the likelihood of each input belonging to the target classes.

ISLab

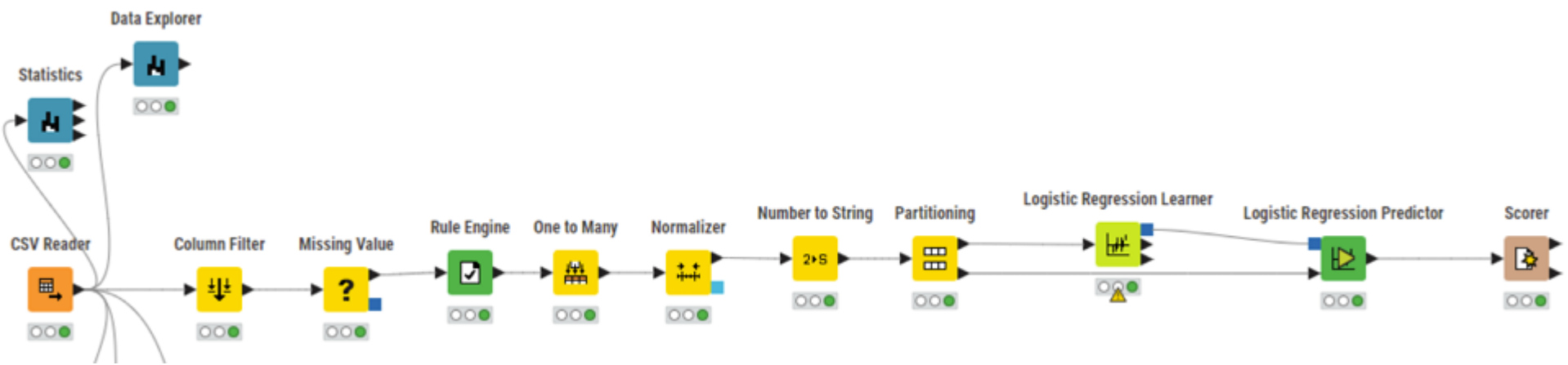
Logistic Regression



Logistic Regression

To assess the performance of the model, in the Node Repository, search for the **Scorer** node.

Connect the **output** port of the **Logistic Regression Predictor** node to the **input** port of the **Scorer** node.



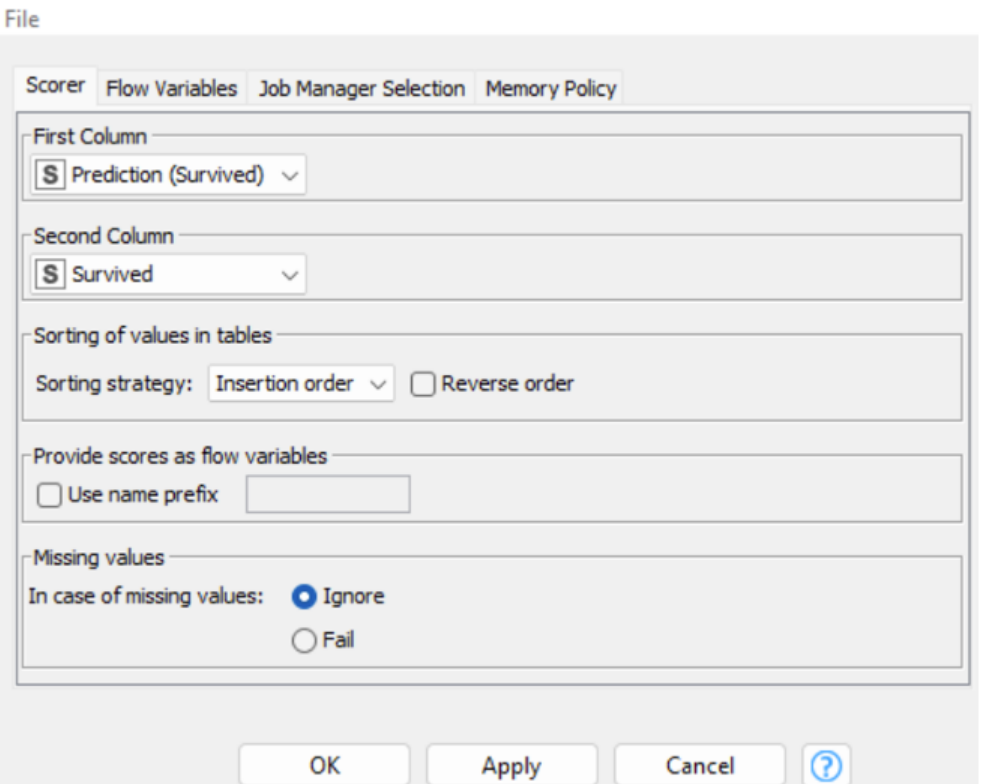
Logistic Regression

Open the configuration panel of the Scorer node.

Select **Prediction (Survived)** as the **First Column** and **Survived** as the **Second Column.**

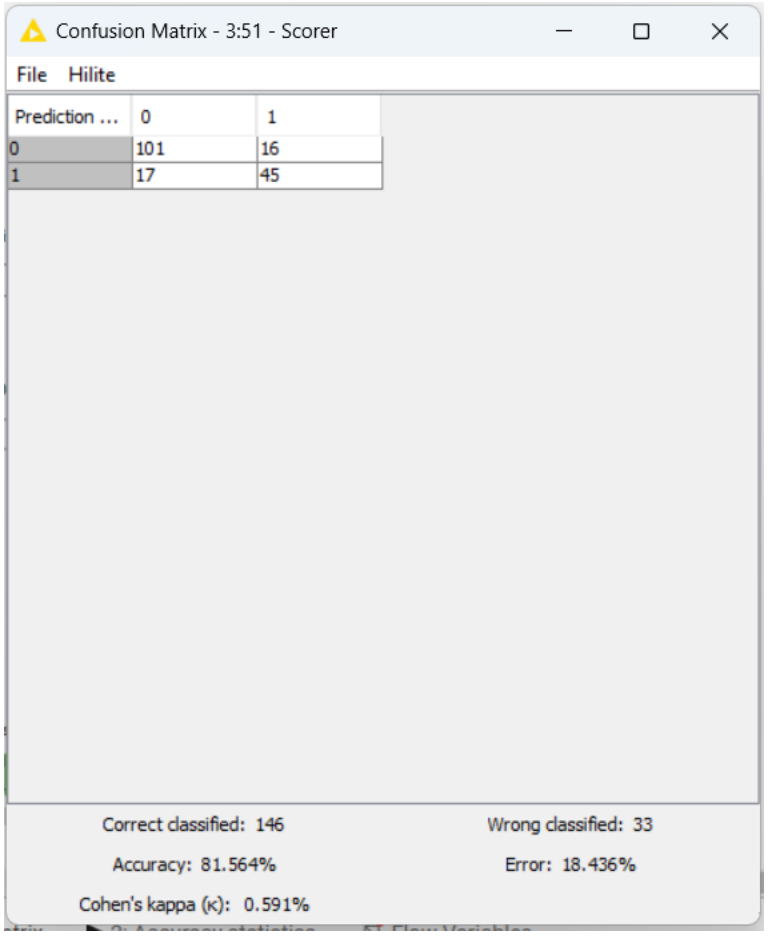
Run the node.

Click on the magnifying glass icon of the Scorer node to access the **performance metrics.**



Logistic Regression

Let’s start by analyzing the **confusion matrix:**



- **True Negatives (0 predicted as 0):** 101 passengers who did not survive were correctly classified as not surviving.
- **False Positives (0 predicted as 1):** 16 passengers who did not survive were incorrectly classified as surviving.
- **False Negatives (1 predicted as 0):** 17 passengers who survived were incorrectly classified as not surviving.
- **True Positives (1 predicted as 1):** 45 passengers who survived were correctly classified as surviving.

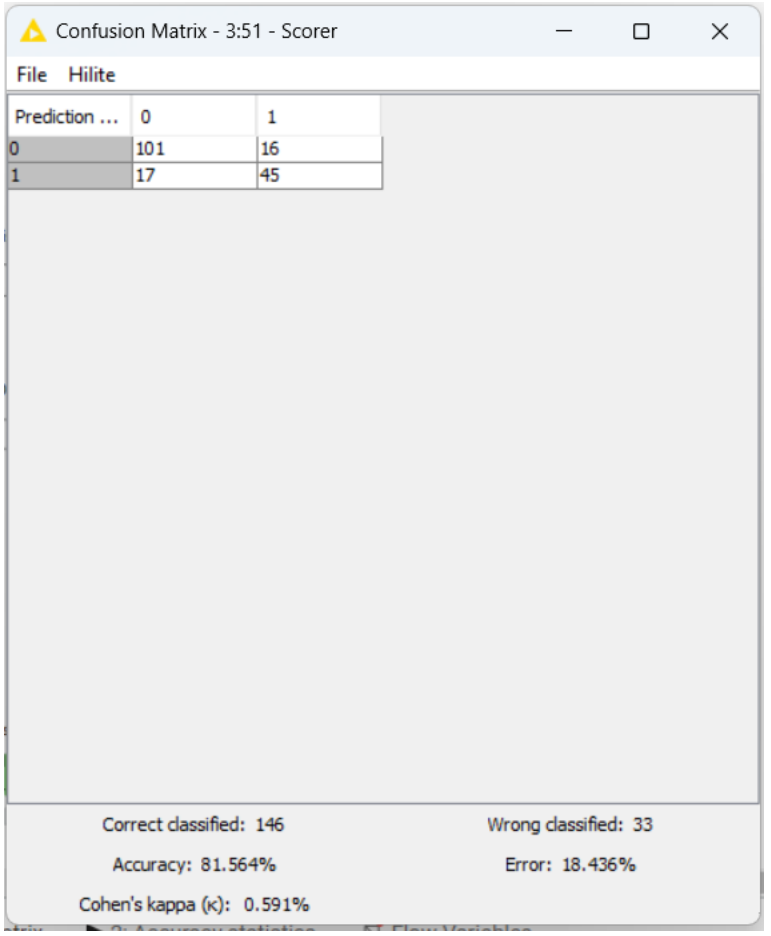
Logistic Regression

In terms of **performance metrics:**

• **Accuracy:** The model achieved an accuracy of 81.56%, meaning it correctly classified about 82% of the passengers.

• **Error Rate:** The model has an error rate of 18.44%, representing the percentage of passengers that were misclassified.

• **Cohen's Kappa (K):** 0.591 indicates moderate to substantial agreement between the predicted and actual classifications, adjusting for chance agreement. A higher K value (closer to 1) would indicate stronger predictive power.



ISLab

Logistic Regression

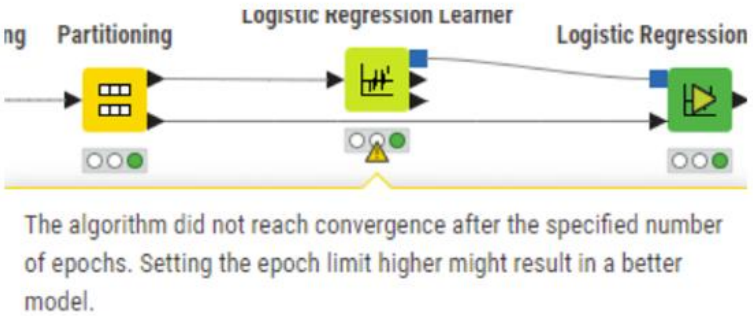
The model generally **performs well,** with **a relatively high accuracy** and a moderate Cohen's Kappa.

However, there is still a **noticeable misclassification rate**, particularly with 17 passengers who actually survived but were predicted not to survive. Improving feature selection, tuning the model, or using additional preprocessing steps could help reduce these errors, especially the false negatives, to improve the model's sensitivity to passengers who survived.

Logistic Regression

If you hover the Logistic Regression Learner, you see a danger sign with the following message:

”The algorithm did not reach convergence after the specified number of epochs. Setting the epoch limit higher might result in a better model.”



A single iteration through the training dataset is called an epoch.

Logistic Regression

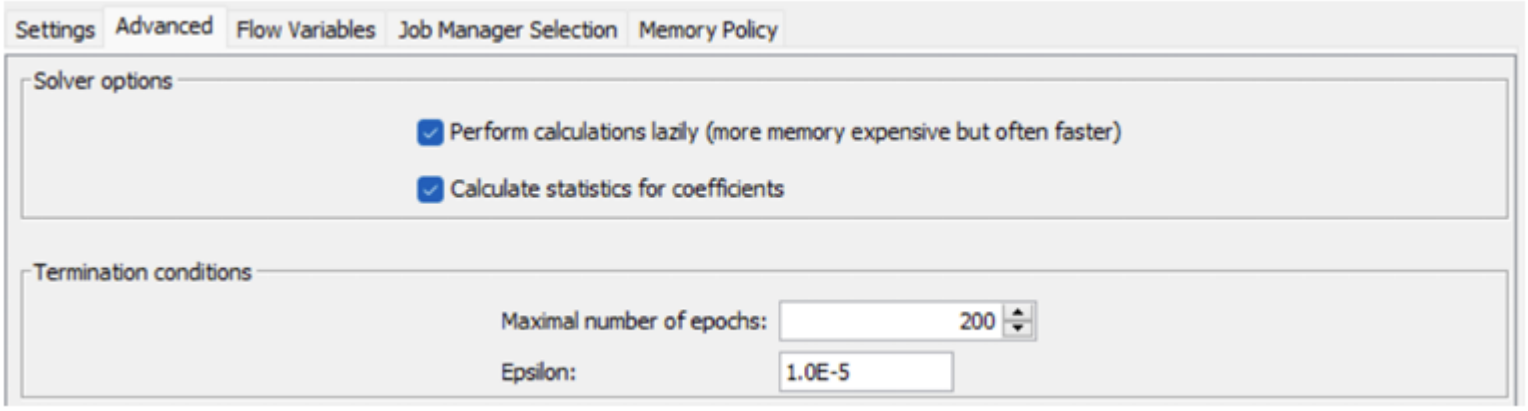
This message from the Logistic Regression Learner node in KNIME indicates that **the algorithm was unable to reach convergence within the set number of epochs, or iterations, that it performs during the training process.** In logistic regression, convergence means that the model's parameters have stabilized and are no longer changing significantly from one iteration to the next.

Increasing the epoch limit allows the algorithm more time to find the best fit for the data. This adjustment can help the model achieve better convergence, potentially improving classification accuracy and reducing error rates. In this case, raising the epoch limit could lead to a more accurate logistic regression model that may perform better in classifying passengers as ”Survived” or ”Not Survived.”

Logistic Regression

Open the Logistic Regression Learner node's configuration panel.

Select the **Advanced** option and change the number of epochs from **100 to 200**.



Logistic Regression

Run all the remaining nodes.

The danger sign has been resolved, and the model has reached convergence with consistent performance metrics, that correspond to the same obtained as previously. This can be due to the fact that some models may have limited capacity to learn complex relationships in the data. Increasing epochs won’t enhance performance if the model architecture cannot capture the necessary complexity.

Logistic Regression

Check performance metrics again!

Universidade do Minho
Escola de Engenharia
Departamento de Informática

KNIME

Pre-Processing + Linear Regression

MESTRADO EM ENGENHARIA DE TELECOMUNICAÇÕES E INFORMÁTICA
Inteligência Artificial para as Telecomunicações
2025/2026